## 1. ¿Qué es el proyecto?

Es una billetera virtual para el celular. La persona se registra y puede depositar, retirar, enviar dinero a otra cuenta y pagar servicios (luz, agua, internet). Si alguien le envía dinero mientras tiene la app abierta, el saldo cambia en el momento, sin recargar.

El proyecto tiene dos partes que se comunican por red:

| Parte | Con qué está hecha | Qué hace |
|---|---|---|
| App móvil (carpeta `app`) | Kotlin, Jetpack Compose, Room, Retrofit, OkHttp, corrutinas | Muestra las pantallas, guarda el historial en el celular y habla con el servidor. |
| Servidor (carpeta `backend`) | Node.js, Express, ws, SQLite | Guarda los usuarios y los saldos, valida cada operación y avisa en tiempo real. |

El saldo real vive en el servidor. La app solo lo muestra. Por eso dos celulares distintos pueden enviarse dinero entre sí.

## 2. Arquitectura general

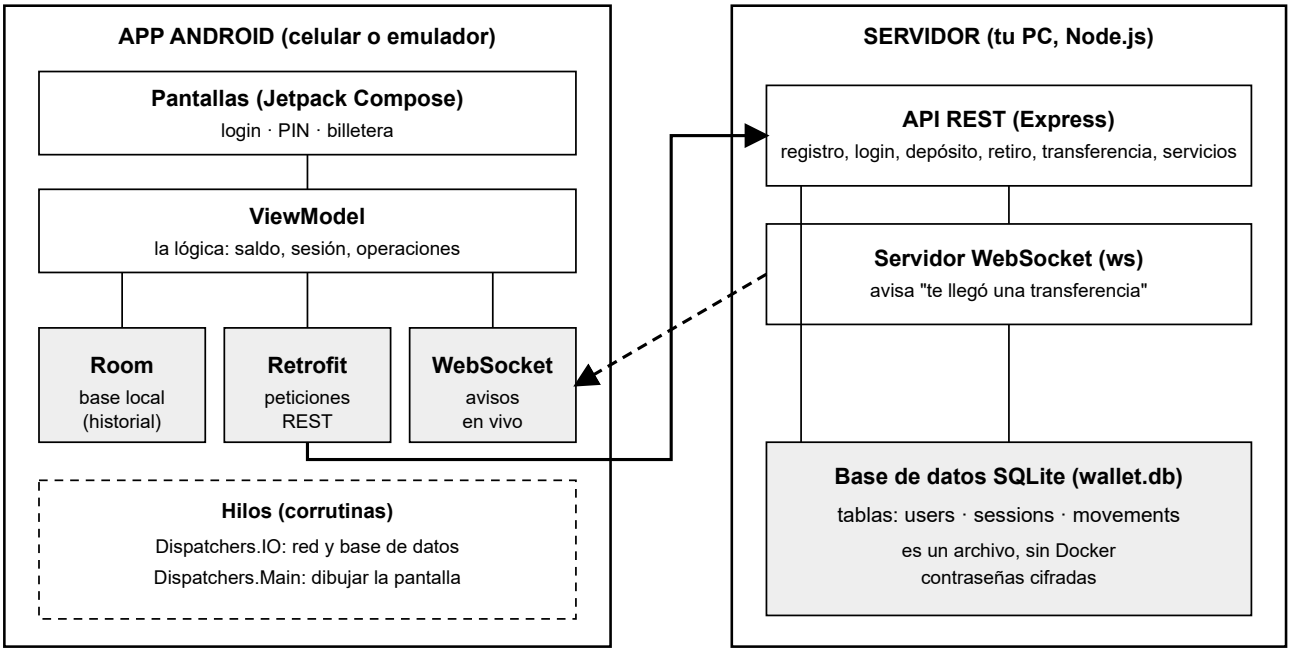



Figura 1. Línea continua: REST (HTTP + JSON), la app pide y el servidor responde. Línea punteada: WebSocket, el servidor avisa a la app.

**REST y WebSocket: para qué se usa cada uno**

| | REST | WebSocket |
|---|---|---|
| Cómo funciona | La app pregunta y el servidor responde. Cada petición es independiente. | Se abre una conexión que queda abierta. El servidor puede escribirle a la app cuando quiera. |
| Quién empieza | Siempre la app. | La app abre la conexión, y luego el servidor manda los avisos. |
| Se usa para | Registrarse, entrar, depositar, retirar, transferir, pagar. | Avisar al receptor que le llegó una transferencia. |

Si solo existiera REST, la app tendría que preguntar cada pocos segundos "¿me llegó algo?". Eso se llama polling y la rúbrica lo penaliza. Con WebSocket el servidor avisa solo, al instante.

## 3. Cómo está organizada la app

| Archivo | Para qué sirve |
|---|---|
| `MainActivity.kt` | Es la entrada. Decide qué pantalla mostrar: login, PIN o billetera. |
| `WalletViewModel.kt` | Guarda el estado (saldo, sesión, conexión), hace las operaciones y maneja el PIN. |
| `ui/AuthScreen.kt` | Pantalla de ingresar y crear cuenta. |
| `ui/PinScreen.kt` | Crear y usar el PIN de 4 dígitos. |
| `ui/WalletScreen.kt` | Pantalla principal: saldo, estado de conexión, 4 botones e historial. |
| `data/local/Local.kt` | Room: la tabla local del historial. |
| `data/remote/Remote.kt` | Retrofit (REST) y el WebSocket, que se reconecta solo. |

Se usó el patrón MVVM: las pantallas solo dibujan lo que el ViewModel les dice. Nunca hablan directo con la red ni con la base de datos. Así la lógica queda separada del diseño.

### Hilos y corrutinas

El hilo principal solo debe dibujar la pantalla. Si hace una llamada de red ahí, la app se congela o se cierra. Por eso:

- **Dispatchers.IO:** todas las llamadas REST, el WebSocket y las lecturas y escrituras de Room.
- **Dispatchers.Main:** solo para actualizar el saldo y la lista cuando ya llegó el resultado.

Una corrutina es una tarea que puede esperar la respuesta de la red sin bloquear el hilo. Con `withContext(Dispatchers.IO)` se pasa al hilo de fondo y se vuelve solo al terminar.

```
// Ejemplo resumido de WalletViewModel
viewModelScope.launch {                        // empieza en Main
    val res = withContext(Dispatchers.IO) {    // pasa a IO
        val r = api.deposit(AmountReq(monto))  // 1) llamada REST
        dao.insert(movimiento)                 // 2) guarda en Room
        r
    }
    setBalance(res.balance)                    // vuelve a Main y actualiza el saldo
}
```

El historial es reactivo: Room entrega un `Flow` y cada vez que se inserta una fila, la lista se redibuja sola. Por eso el saldo y la lista cambian sin recargar.

## 4. Cómo funcionan las operaciones

### 4.1 Registro y login

1. La persona escribe nombre, correo y contraseña. La app revisa que el correo sea válido y que la contraseña tenga 8 o más caracteres, con mayúscula, minúscula y número.
2. La app envía `POST /api/auth/register`. El servidor revisa todo otra vez (no confía en la app), cifra la contraseña, guarda al usuario y le asigna un número de cuenta como `ACC-4821`.
3. El servidor responde con un token, que es una cadena aleatoria larga. La app lo guarda y lo manda en cada petición: `Authorization: Bearer token`.
4. Con el token el servidor sabe quién es la persona sin pedirle la contraseña de nuevo.

### 4.2 Transferencia entre dos usuarios

Ejemplo: Melanie le envía $50 a Carlos. Carlos tiene la app abierta.

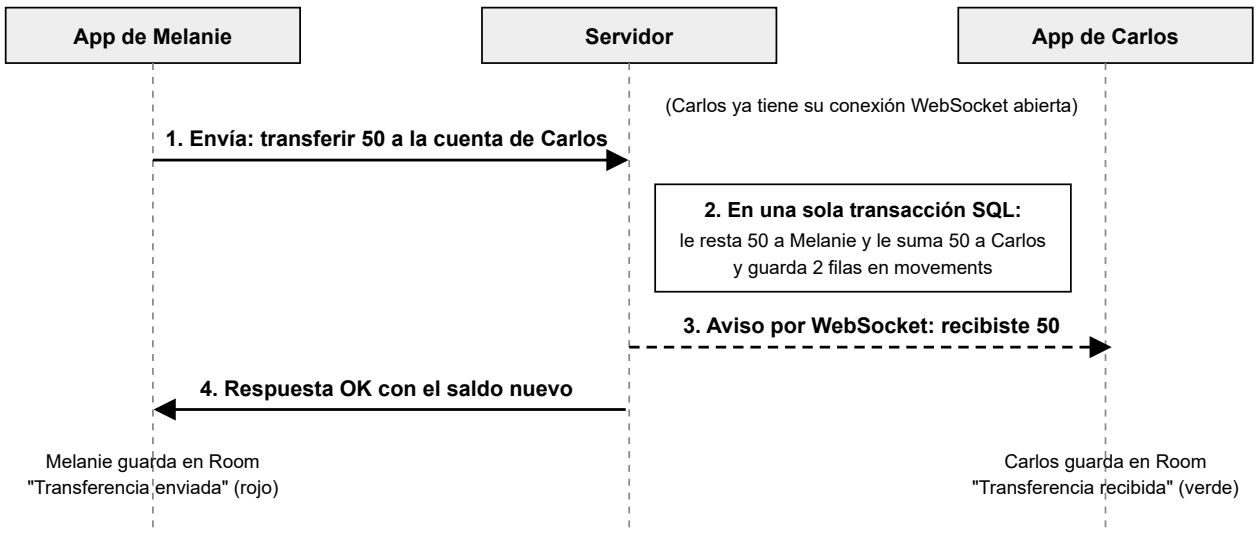



Figura 2. El que envía usa REST. El que recibe se entera por WebSocket sin hacer nada.

Antes de mover el dinero, el servidor revisa que el monto sea válido, que la cuenta destino exista (si no, error 404), que no sea la misma cuenta y que haya saldo suficiente (si no, error 409 "Saldo insuficiente"). La app muestra ese mensaje sin cerrarse.

### 4.3 Pago de servicios

La persona escribe un código de contrato y un monto. El servidor acepta códigos que empiecen con `LUZ-`, `AGUA-` o `NET-` seguidos de números, por ejemplo `LUZ-123456`. Descuenta el saldo y responde qué servicio era. La app lo guarda en Room con la categoría "Servicio Básico" y un icono distinto (rayo, gota o wifi).

### 4.4 El WebSocket

- Al entrar, la app abre `ws://servidor/ws?token=...`. El servidor revisa el token y anota ese socket a nombre de la cuenta.
- Si la conexión se cae, la app reintenta cada 3 segundos. El círculo de la cabecera pasa de verde ("En línea") a rojo ("Desconectado") y vuelve.

- Cada 15 segundos manda un ping para mantener la conexión viva.

## 5. Seguridad

| Qué | Cómo se hizo | Para qué |
|---|---|---|
| Contraseñas | Se guardan con scrypt y una sal distinta por usuario. Nunca en texto plano. | Si roban la base de datos, no pueden leer las contraseñas. |
| Regla de contraseña | Mínimo 8 caracteres, con mayúscula, minúscula y número. Se valida en la app y en el servidor. | Evitar claves fáciles. El servidor es el que realmente protege. |
| Sesión (token) | 32 bytes aleatorios. En la base solo se guarda su hash SHA-256. | Aunque se filtre la tabla `sessions`, no sirve para entrar. |
| PIN de 4 dígitos | Se guarda en el celular con PBKDF2 y sal. A los 5 intentos fallidos se borra y pide la contraseña. | Entrar rápido, y proteger el saldo si alguien toma el celular. |
| Quién hace la operación | El servidor usa la cuenta del token, no una que mande la app. | Nadie puede mover dinero de otra cuenta cambiando un dato. |
| Consultas SQL | Son preparadas (usan `?` en vez de pegar texto). | Evita la inyección SQL. |

Limitaciones, por si preguntan:

- La comunicación va por HTTP, no HTTPS. En producción se usaría HTTPS. Aquí es una red local de pruebas.
- Los tokens no caducan solos.
- El PIN se pide solo al abrir la app de cero, no al volver de segundo plano.
- Los montos se guardan como decimal (`REAL`). En un banco real se usarían enteros en centavos para evitar errores de redondeo.

# 6. La base de datos relacional

## 6.1 Qué significa "relacional"

Una base relacional guarda la información en tablas, parecidas a hojas de Excel con columnas fijas, y conecta las tablas entre sí con identificadores en lugar de repetir datos.

- **Tabla:** un tipo de cosa (usuarios, movimientos).
- **Fila:** un registro concreto (un usuario, un depósito).
- **Clave primaria (PK):** el número que identifica cada fila, sin repetirse (por ejemplo `id`).
- **Clave foránea (FK):** una columna que apunta a la clave primaria de otra tabla. Así se crea la relación.

Ejemplo: en `movements` no se repite el nombre ni el correo de la persona en cada depósito. Solo se guarda `user_id = 3`, que apunta al usuario 3 de la tabla `users`. Si el usuario cambia su nombre, se cambia en un solo lugar.

## 6.2 Las tablas del servidor (SQLite, archivo `backend/wallet.db`)

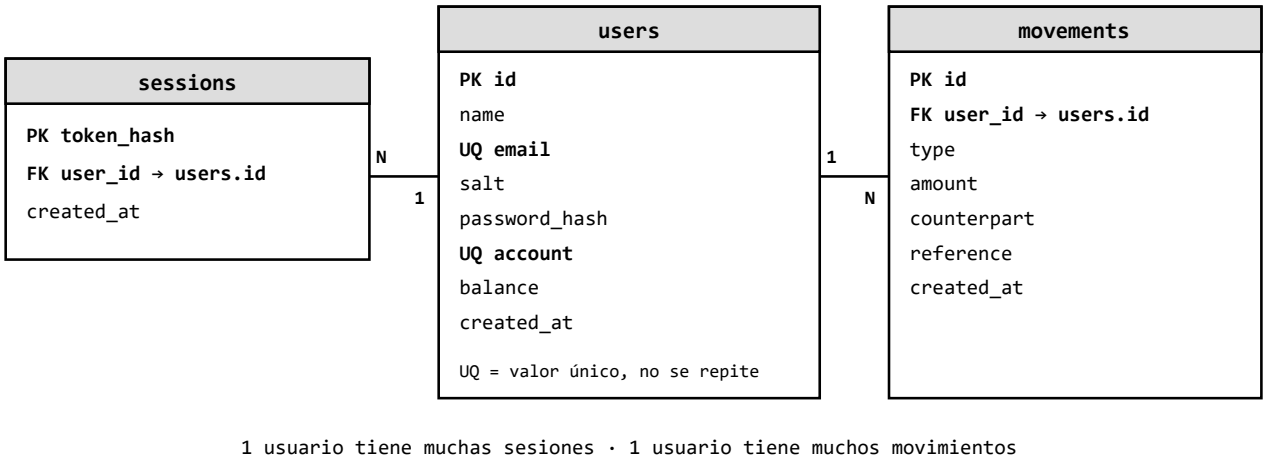



Figura 3. Diagrama de las tablas. Las relaciones son de uno a muchos (1:N).

| Tabla | Qué guarda | Detalles |
|---|---|---|
| `users` | Una fila por persona registrada. | `email` y `account` son únicos: la base impide dos usuarios con el mismo correo. `balance` es el saldo actual. `salt` y `password_hash` forman la contraseña cifrada. |
| `sessions` | Una fila por cada login activo. | `user_id` es la clave foránea hacia `users`. Permite saber a quién pertenece cada token. |
| `movements` | El registro de todo lo que pasó con el dinero. | `type` puede ser DEPOSIT, WITHDRAW, TRANSFER_OUT, TRANSFER_IN o SERVICE. `reference` es un código del movimiento. |

## 6.3 Las relaciones

- **users 1 a N sessions:** un usuario puede tener varias sesiones (celular y emulador a la vez), pero cada sesión es de un solo usuario.

- **users 1 a N movements:** un usuario tiene muchos movimientos, y cada movimiento es de un solo usuario.
- **Una transferencia genera 2 filas** en `movements`: una `TRANSFER_OUT` para quien envía y una `TRANSFER_IN` para quien recibe. Las dos comparten la misma `reference`, y en `counterpart` cada una guarda la cuenta de la otra persona.

Ejemplo: Melanie (id 1, cuenta ACC-1111) le envía $50 a Carlos (id 2, cuenta ACC-2222):

| id | user_id | type | amount | counterpart | reference |
|---|---|---|---|---|---|
| 10 | 1 (Melanie) | TRANSFER_OUT | 50 | ACC-2222 | TRF-ABC01 |
| 11 | 2 (Carlos) | TRANSFER_IN | 50 | ACC-1111 | TRF-ABC01 |

### 6.4 Consultas para mostrar en DBeaver

```
-- 1) Usuarios (la contraseña sale cifrada, no se puede leer)
SELECT id, name, email, account, balance, password_hash FROM users;

-- 2) JOIN: movimientos junto al nombre de su dueño
SELECT u.name, m.type, m.amount, m.counterpart, m.reference
FROM movements m
JOIN users u ON u.id = m.user_id
ORDER BY m.id DESC;

-- 3) Cuántos movimientos tiene cada usuario
SELECT u.name, COUNT(m.id) AS total
FROM users u LEFT JOIN movements m ON m.user_id = u.id
GROUP BY u.id;

-- 4) Las dos filas de una misma transferencia
SELECT * FROM movements WHERE reference = 'TRF-ABC01';
```

El JOIN es lo central de lo relacional: junta filas de dos tablas usando la clave foránea (`m.user_id = u.id`).

### 6.5 Transacciones

Una transferencia son varios cambios: restar al que envía, sumar al que recibe y guardar 2 movimientos. Si el servidor se cae a la mitad, podría desaparecer dinero. Por eso todo va dentro de una transacción: o se hace todo, o no se hace nada.

```
BEGIN;
  UPDATE users SET balance = (saldo de Melanie - 50) ...;
  UPDATE users SET balance = (saldo de Carlos + 50) ...;
  INSERT INTO movements (... TRANSFER_OUT ...);
  INSERT INTO movements (... TRANSFER_IN ...);
COMMIT;    -- si algo falla se hace ROLLBACK y no cambia nada
```

Esto se llama atomicidad y es una de las ventajas de usar una base relacional.

### 6.6 La segunda base de datos: Room en el celular

La app tiene su propia base SQLite local, hecha con la librería Room. Sirve para ver el historial sin internet y para que no se pierda al cerrar la app.

| Columna | Qué es |
| --- | --- |
| `id` | Clave primaria autonumérica. |
| `owner` | Número de cuenta dueño del movimiento. Hace que cada usuario vea solo su historial, aunque dos personas usen el mismo celular. |
| `type`, `amount`, `timestamp` | Tipo (depósito, retiro...), monto y fecha. Con eso se pinta en verde o rojo. |
| `counterpart`, `category`, `detail`, `reference` | Cuenta de la otra persona o contrato; "Servicio Básico"; Luz, Agua o Internet; código del movimiento. |

| Servidor (SQLite en Node) | Celular (Room) |
| --- | --- |
| Es la fuente de verdad: saldos reales, usuarios y contraseñas. Tiene varias tablas unidas por claves foráneas. | Es una copia para mostrar el historial. Tiene una sola tabla. `owner` apunta a la cuenta, pero no es una clave foránea real porque la tabla `users` está en la otra base. |

Son dos bases porque el celular y el servidor son máquinas distintas. Por eso el proyecto es "distribuido": cada una guarda lo suyo y se mantienen al día por REST y WebSocket.

## 7. Cómo probar que funciona

1. Servidor: `cd backend`, luego `npm install` y `npm start`. Debe decir `Backend en http://0.0.0.0:3000`.
2. App: abrir la carpeta en Android Studio y dar Run en el emulador. La cabecera debe decir "En línea".
3. Registro: crear una cuenta con una contraseña como `Abcdefg1`. Arriba aparece el número de cuenta.
4. Operaciones: depositar 100 (sube el saldo y sale una fila verde). Retirar más de lo que hay (sale "Saldo insuficiente").
5. Tiempo real: en Postman importar `backend/postman_collection.json`, poner tu cuenta en la variable `destino` y correr los 3 pasos. El saldo cambia solo en la app.
6. Persistencia: cerrar la app del todo y abrirla. El historial sigue ahí (Room).
7. Base de datos: abrir `backend/wallet.db` en DBeaver y correr las consultas de la sección 6.4.

## 8. Preguntas posibles en la defensa

| Pregunta | Respuesta corta |
|---|---|
| ¿Dónde corren las llamadas de red? | En `Dispatchers.IO`, nunca en el hilo principal. Así no hay NetworkOnMainThreadException ni ANR. |
| ¿Cómo se actualiza el saldo del receptor sin recargar? | Por WebSocket. El servidor manda un evento `TRANSFER_IN`, la app lo guarda en Room y actualiza un `StateFlow` que redibuja la pantalla. |
| ¿Por qué Room y no SQLite a mano? | Room es una capa sobre SQLite. Revisa las consultas al compilar, usa DAOs y entrega `Flow` para que la lista sea reactiva. |
| ¿Qué pasa si no hay internet? | El historial sigue visible. Las operaciones muestran "No se pudo conectar con el servidor" sin cerrar la app, y el WebSocket reintenta cada 3 segundos. |
| ¿Cómo guardas las contraseñas? | Con scrypt y una sal por usuario. Solo se guarda el hash. |
| ¿Qué es una clave foránea? | Una columna que apunta a la clave primaria de otra tabla, como `movements.user_id` hacia `users.id`. |
| ¿Por qué SQLite y no MySQL o PostgreSQL? | Es un archivo, no necesita instalación ni Docker, y alcanza para este proyecto. Se puede abrir con DBeaver. |
| ¿Qué mejorarías? | HTTPS, que los tokens caduquen, montos en centavos enteros, y una base como PostgreSQL si hubiera muchos usuarios a la vez. |